Human-Computer Interaction - Camera Mouse (Marker-Based Gesture Control)

Student: Enis Ögdüm **Date:** 2026-01-14

1. Introduction

The objective of this laboratory exercise was to explore the principles of touchless interaction by implementing and evaluating a marker-based computer vision system. The goal was to control the mouse pointer using a webcam and a color-tracked object, simulating a "Camera Mouse." This report documents the experimental setup, the implementation of the tracking mechanism, and an analysis of the environmental and physical factors influencing system performance.

2. Methodology and Experimental Setup

The experiment utilized the "Touchless" demo software, which employs a color-segmentation algorithm to detect and track a specific object in the video feed. - **Hardware**: A standard webcam (resolution: 640x480, ~26 fps) and a PC monitor. - **Marker**: A blue test tube cap was selected as the control object due to its distinctive hue and high saturation. - **Environment**: A university computer lab with artificial overhead lighting.

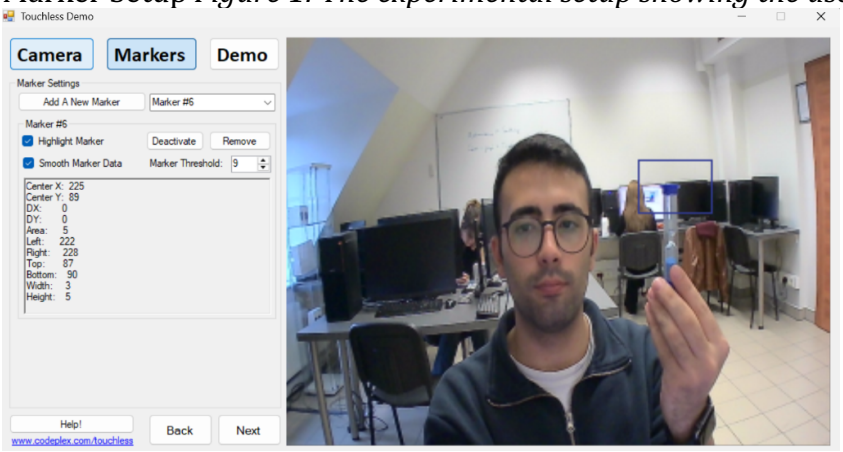
3. Exercise 1: System Implementation and Evaluation

3.1 Marker Selection and Detection

The primary mechanism for tracking is color thresholding (likely operating in HSV/HSL color space). The blue test tube cap was introduced to the camera, and the software was calibrated to recognize its specific color signature.

Figure 1 illustrates the user positioning the marker. The system provided real-time feedback by drawing a bounding box around the detected cluster of pixels.

Marker Setup *Figure 1: The experimental setup showing the user holding the blue marker*



3.2 Tracking Accuracy

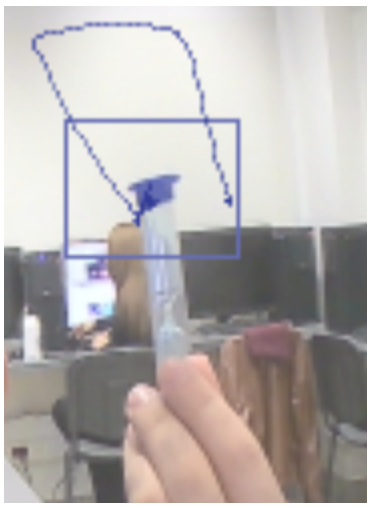
The system successfully isolated the blue marker from the background. *Figure 2* demonstrates the segmentation mask, where the specific blue shade of the cap is highlighted, separating it from the user's hand and the background.

Marker Detection Closeup *Figure 2: Successful segmentation of the blue marker.*

3.3 Control Demonstration

To verify control authority, a drawing test was conducted. By moving the marker in the air, the user was able to guide the cursor to draw vertical lines (*Figure 4*). The mapping between the physical marker position and the on-screen cursor was direct (absolute positioning).

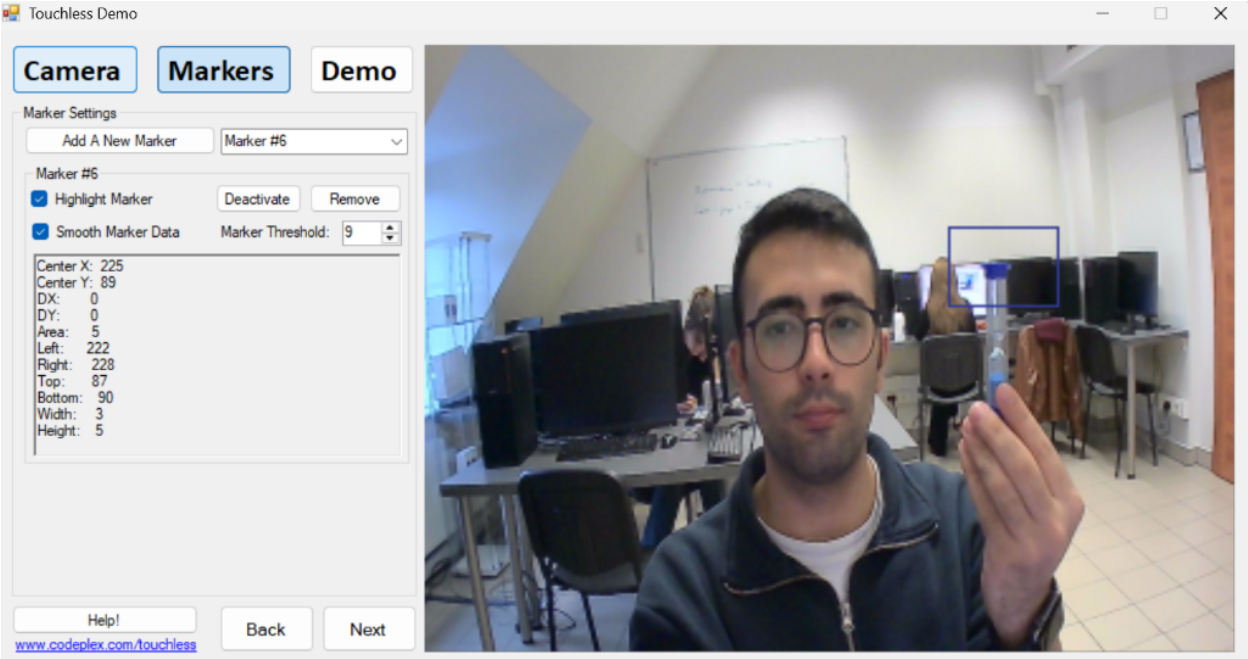
Tracking Test *Figure 4: A trajectory test demonstrating vertical cursor control using the marker.*



3.4 Application Evaluation (Snake Game)

The system's usability was further tested using the "Snake" game provided in the Demo tab (*Figure 5*). - **Effectiveness**: The game requires rapid, precise turns. While the system could detect the direction of movement, the inherent latency (delay) of the webcam processing pipeline made the game difficult to play at higher speeds. - **Observation**: A "drift" effect was noticed; stopping the physical marker did not always result in an immediate stop of the cursor due to smoothing algorithms applied to the raw coordinates.

Demo Tab *Figure 5: The demo interface identifying the marker's coordinates for application control.*



4. Exercise 2: Analysis of Tracking Factors

A series of experiments were conducted to determine the robustness of the computer vision algorithm. The following factors were identified as critical to performance:

4.1 Color and Background

Observation: The tracking relies heavily on color uniqueness. - **Analysis**: The blue marker worked effectively because the background (white walls, beige desks, dark monitors) lacked significant blue components. When a blue object (e.g., a piece of clothing) entered the frame, the centroid calculation shifted towards the average position of all blue pixels, causing cursor instability (false positives).

4.2 Lighting Conditions

Observation: Stability degraded in shadowed areas. - **Analysis**: Color detection algorithms are sensitive to illumination changes. In low light, the saturation and value components of the marker's color decrease, often falling outside the calibrated threshold range. Consistent, diffuse lighting is required for robust tracking.

4.3 Speed of Movement

Observation: Rapid movements caused tracking loss. - **Analysis**: High-speed motion introduces "motion blur," where the marker's color blends with the background pixels in the video frame. This blending alters the pixel values, causing them to fail the color threshold check.

4.4 Shape and Size

- **Shape**: The algorithm appears to be blob-based, ignoring the geometric shape of the object. Rotating the cylindrical cap (changing its perceived 2D shape) had little effect on detection as long as the color mass remained visible.
- **Size**: There is a minimum and maximum effective size.
  - **Too Small**: If the marker moves too far away, the "blob" falls below the pixel area threshold (noise filter) and is ignored.
  - **Too Large**: If brought too close (e.g., <20cm), the object fills the screen, making coordinate calculation (center of mass) meaningless.

4.5 Physiological Markers (Hand/Face)

Attempts to use a hand or face as a marker yielded poor results compared to the artificial marker. - **Analysis**: Skin tone is not a uniform color; it contains complex gradients of shadows and highlights. Furthermore, skin colors (beiges/browns) often resemble background elements (e.g., wooden desks, walls), leading to poor segmentation and "jittery" control.

5. Conclusion

The experiments demonstrate that a single-color marker-based system is a viable, low-cost solution for touchless HCI. However, it is strictly limited by environmental constraints. For a robust real-world application, the system would require: 1. **Adaptive Lighting Correction**: To handle shadows. 2. **Predictive Filtering (e.g., Kalman Filter)**: To reduce latency and handle motion blur during rapid movements. 3. **Background Subtraction**: To isolate the user from a cluttered environment